

UNIVERSITY PARTNER



Student Name: Rashi Bhatta

Student ID: 2417499

Group: The Matrix | Section: L6CG12

Module Leader: Mr. Siman Giri | Tutor: Ms. Jinu Nyachhyon

Submitted on :May 10, 2026

Herald College Kathmandu | University of Wolverhampton

LONG QUESTION [5 Marks] — Answer Any 1	2
Question 1: Unsupervised Learning at a Digital Payment Platform (eSewa)	2
Problem 1: Grouping Users by Spending Behaviour (Customer Segmentation)	2
SHORT QUESTION — Category 1: Overfitting [2.5 Marks]	6
Question 1: Overfitting and Underfitting — What They Mean and Why They Matter	6
What is Overfitting?	6
What is Underfitting?	6
How Do You Tell the Difference?	6
Why Do Both Problems Hurt the Model?	7
How Weights Are Reused	9
Where Each Architecture Gets Used	9
Training Challenge 1 — Vanishing and Exploding Gradients	9
Training Challenge 2 — Overfitting	10

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Module: 6CS012 | Final Integrated Assessment

Part I — Question and Answer Report

Assessment Component	Part I — QnA (10 Marks)
Long Question Answered	Q1 — Unsupervised Learning at eSewa
Short Q1 Answered	Q1 — Overfitting and Underfitting
Short Q2 Answered	Q1 — CNN vs RNN Architecture
Submission Format	Individual Written Report (DOCX)

LONG QUESTION [5 Marks] — Answer Any 1

Question 1: Unsupervised Learning at a Digital Payment Platform (eSewa)

eSewa is the most widely used digital payment platform in Nepal, handling millions of financial transactions every single day. Almost all of this data comes without any labels, which makes it impossible to use supervised machine learning methods directly. Unsupervised learning is different — it does not need labeled data and instead picks up patterns that are naturally present in the data. Two key areas where this kind of learning can genuinely help eSewa are discussed below.

Problem 1: Grouping Users by Spending Behaviour (Customer Segmentation)

Right now, eSewa treats all of its users the same way when it comes to marketing, app features, and notifications. But the truth is that users behave very differently from one another. Some people only open the app once or twice a month to pay electricity or water

bills. Others make small payments at local shops almost every day. And then there is a group that mostly uses the app to send money to family members. When the same offer or message goes out to all of these people, it simply does not work well for most of them, and the company ends up wasting marketing money on people who are not interested. Clustering is the right kind of machine learning to solve this. K-Means is a good starting point — it sorts users into groups by looking at how similar their behaviour patterns are. The useful data points to feed into the model include how often someone makes a transaction, how much they usually spend, what type of payments they make (bills, shops, or transfers), and what time of day they are usually active. Once the algorithm finishes, every user gets placed into a group that contains other users who behave similarly. For example, one group might be made up of people who make lots of small purchases, while another group might be people who rarely use the app but spend a large amount when they do.

When real financial data does not fall into neat, round clusters — which happens quite often — DBSCAN works better. This algorithm does not need you to decide the number of groups beforehand. It finds concentrations of similar users on its own and labels anyone who does not clearly belong to any group as an outlier. These outlier users are actually worth looking at closely because they often show unusual spending patterns that might be worth investigating further.

Once the groups have been created, the results are saved against each user's account and updated regularly — for example, every week — using a scheduling tool like Apache Airflow. Marketing teams can then send very specific campaigns to each group instead of blasting the same message to everyone. Users who pay a lot of bills could be offered cashback on utility payments, while users who frequently send money to others might appreciate a promotion on transfer fees. This targeted approach generally leads to better results and costs less than untargeted advertising.

One known issue with K-Means is that you must tell it how many groups to create before it runs. Choosing too few groups means very different kinds of users get mixed together. Choosing too many means the groups become so small and specific that they are hard to act on. The Elbow Method helps here — it shows a chart of how the variation inside

clusters changes as you try different numbers of groups, and the right number is usually where the chart starts to level off. That said, as eSewa's user base grows and spending habits shift — especially around major festivals like Dashain or Tihar — the ideal number of groups might need to change, which means the whole process has to be run again from time to time.

Problem 2: Spotting Suspicious Transactions Through Anomaly Detection

Fraud in a digital payment system can show up in many ways — someone logging in from an unusual location and immediately making a large transfer, the same small payment being made repeatedly in a short window, or money being sent to an account that has never been used before. The big challenge with fraud detection is that actual fraud cases make up only a tiny fraction of all the transactions happening every day. Because of this, there is not enough labeled fraud data to train a supervised model reliably. The system instead has to learn what normal looks like and then raise a flag whenever something does not match that pattern.

An Autoencoder is a good fit for this kind of problem. It is an unsupervised neural network that works by compressing transaction data down to a small internal summary and then trying to rebuild the original data from that summary. When it is trained only on genuine, normal transactions, it gets very good at rebuilding them accurately. But when a suspicious or fraudulent transaction goes through the same network, the reconstruction does not match well, producing a high error score. Any transaction with an error score above a set limit gets flagged for a closer look.

If the priority is speed and ease of use rather than the most powerful detection, Isolation Forest is a lighter option. It works by repeatedly cutting the data into random sections and checking how quickly individual transactions get separated from the rest. Fraudulent ones are unusual by nature, so they tend to get isolated much faster than ordinary transactions. This makes the algorithm fast and fairly effective without requiring a lot of setup or training time.

The detection system can be plugged directly into a live transaction stream using a tool like Apache Kafka. Every payment that comes in gets scored by the model right away. If the score goes above the threshold, the transaction is paused and sent into a queue for a human fraud analyst to check. At the same time, the account holder gets an automatic message letting them know their transaction has been held. This setup cuts down the time between when fraud happens and when it gets caught, and it also reduces the amount of manual checking that the fraud team has to do.

The main weakness of this kind of model is that it can only identify what looks unusual compared to what it learned during training. Festival periods in Nepal — like Dashain and Tihar — bring a big surge in transaction volumes, larger-than-usual payment amounts, and transfers to merchants that users do not normally interact with. If the model was not retrained to account for this new normal, it will start marking lots of completely legitimate payments as suspicious. This slow mismatch between what the model thinks is normal and what is actually normal is called concept drift. It means the model needs to be retrained on a regular schedule and monitored continuously so it stays useful over time.

SHORT QUESTION — Category 1: Overfitting [2.5 Marks]

Question 1: Overfitting and Underfitting — What They Mean and Why They Matter

What is Overfitting?

Overfitting happens when a machine learning model pays too much attention to the training data it was given. Instead of picking up the general rules and patterns that apply broadly, the model ends up memorising the specific details of those training examples — including random noise, minor errors, and rare situations that are unlikely to come up again. The outcome is a model that looks excellent on paper during training but then falls apart when it is tested on data it has never seen before. In short, it learned the examples rather than the actual task.

What is Underfitting?

Underfitting is the opposite problem. It happens when a model is too simple to understand the data properly. An underfit model struggles not just with new data but even with the training data it was already shown, which means the learning process did not really work at all. While overfitting is caused by too much complexity, underfitting happens because the model does not have enough capacity — it simply cannot represent the kind of relationships that exist in the data.

How Do You Tell the Difference?

The easiest way to spot which problem you have is to compare how the model does on training data versus test data. If a model does very well during training but then drops sharply on test data, that is overfitting. If it performs badly on both, that is underfitting. A model that is working well should perform strongly on training data and show only a small, acceptable dip on test data, which means it has genuinely learned the patterns without getting stuck on the specific training examples.

Scenario	Training Performance	Test Performance
Overfitting	Very High	Much Lower
Underfitting	Low	Low
Well-Fitted Model	High	Close to Training

Why Do Both Problems Hurt the Model?

Generalisation is a model's ability to make correct predictions on data it was not trained on. Both of these problems get in the way of that, just in different ways. An overfit model has basically stored the training data instead of learning from it, so even a small difference in how real-world data is presented will throw it off. An underfit model never learned enough to be useful in the first place, so it has nothing reliable to apply to new situations. Either way, the model cannot be trusted in a real deployment.

A Practical Example

Picture a spam filter built on 500 emails. An overfit version of this filter might latch onto specific phrases from the training emails or particular sender addresses. During testing on training data it might score 99% accuracy, but the moment a new spam email arrives written in slightly different language, it misses it entirely and accuracy drops to around 55%. An underfit version might use something overly basic — like flagging any email over 100 words long — without actually learning what makes an email spam. That model might only reach about 60% accuracy in every situation. A well-trained filter sits in the middle: it learns real signals like urgent language and suspicious prize claims without relying too heavily on any single training example.

To fix overfitting, developers commonly use Dropout (randomly turning off some neurons during each training round), Early Stopping (stopping training the moment validation performance stops improving), L2 Regularization (adding a penalty for very large weight values), and Data Augmentation (creating extra training examples by slightly modifying existing ones). For underfitting, the usual fixes are making the model bigger, adding more useful input features, or reducing how much regularization is being applied.

SHORT QUESTION — Category 2: Neural Network Architecture [2.5 Marks]

Question 1: CNN vs RNN — How They Work, Where They Are Used, and What Makes Training Hard

What Kind of Data Each One Handles

A Convolutional Neural Network (CNN) was built for data that has a physical structure — specifically images. A photo is just a large grid of pixels, and nearby pixels tend to belong to the same object or edge. CNNs use small learnable filters that slide across this grid, picking up local patterns like corners, lines, and textures. This makes them very good at understanding visual content. A Recurrent Neural Network (RNN) was designed for sequential data — situations where the order of items matters and what came before affects how you interpret what comes next. Text, speech recordings, stock prices over time, and sensor data all fit into this category.

How Data Moves Through Each Network

In a CNN, information flows in one direction only. It starts at the input image, passes through convolutional layers that detect local visual features, goes through pooling layers that shrink the data while keeping the important parts, and finally reaches fully connected layers that bring everything together to produce a final answer. Each image is handled completely on its own — the CNN has no memory of the images it processed before.

In an RNN, the network goes through the data one step at a time. After each step, it produces a hidden state — a kind of internal memory — that gets carried forward and mixed with the next item in the sequence. By the time the network reaches the end, the hidden state contains a summary of everything that came before it. Every output is shaped by the full history of the sequence up to that point, which is what makes RNNs suited to tasks where context matters.

How Weights Are Reused

CNNs reuse the same filter weights across every part of the image. A filter that has learned to detect a diagonal line will look for that diagonal line everywhere in the image without needing a separate set of weights for each location. This is called spatial weight sharing, and it keeps the number of parameters manageable while also making the network work the same regardless of where a feature appears in the image.

RNNs reuse their weight matrices at every step of the sequence. The same rules that were applied when processing the first word in a sentence are applied when processing the last word. This is called temporal weight sharing and it means the parameter count stays fixed no matter how long the sequence is, though longer sequences do take more time to process.

Where Each Architecture Gets Used

CNN Use Cases	RNN / LSTM Use Cases
Brain scan tumor classification	Filtering spam text messages
Detecting objects in photos	Analysing sentiment in reviews
Reading handwritten digits	Translating languages automatically
Diagnosing medical X-rays	Converting speech into text
Face recognition systems	Predicting stock price movements

Training Challenge 1 — Vanishing and Exploding Gradients

Neural networks learn by backpropagation — a process that works out how much each weight in the network was responsible for the error in the output and then adjusts those weights accordingly. To do this, gradient values get multiplied together as the error signal moves backwards through the layers. In very deep CNNs and in RNNs that process long sequences, this multiplication chain becomes a problem. The gradient can shrink so much with each multiplication that it becomes nearly zero (this is the vanishing gradient problem),

or it can grow so large that it causes chaotic, unstable updates (the exploding gradient problem).

When gradients vanish, the layers closest to the input essentially stop updating — they receive signals so small that nothing meaningful changes. When they explode, the entire training process breaks down. For RNNs, the most widely used fix is the Long Short-Term Memory (LSTM) architecture. LSTMs use gates — a forget gate, an input gate, and an output gate — to carefully control what information gets kept and what gets thrown away at each step. This gating mechanism gives the gradient a clean path through the network even over long sequences. To handle exploding gradients, gradient clipping puts a cap on how large any single update can be. For deep CNNs, Batch Normalization keeps the values between layers in a stable range, which prevents gradients from growing too large or too small as they pass through many layers.

Training Challenge 2 — Overfitting

Large neural networks — including most modern CNNs and RNNs — have a lot of adjustable weights. With so many parameters, the risk of the model fitting the training data too tightly is very real. Instead of learning patterns that hold up across many examples, the network can start memorising specific training cases in a way that does not transfer to new data. Several techniques are used to keep this from happening:

Dropout: During each round of training, a randomly selected group of neurons is switched off temporarily. This stops any single neuron from becoming too important on its own and pushes the network to develop more spread-out representations that hold up better when the input varies slightly.

Early Stopping: A separate validation set is used to track how well the model is doing throughout training. When performance on this set stops improving for a set number of rounds (the patience setting), training is stopped and the best version of the model seen so far is saved. This prevents the model from continuing to fine-tune itself on training-specific details after it has already learned everything that generalises.

Data Augmentation (for CNNs): The training dataset is made larger by creating slightly altered versions of existing images — rotated, flipped, zoomed, or adjusted in brightness.

The model sees each modified image as something new, which forces it to learn features that stay consistent across these changes rather than just memorising exact pixel patterns.

L2 Regularization: A small penalty is added to the loss function based on how large the model's weights are. This pushes the network to avoid relying too heavily on any individual weight, which usually means it is not overfitting to specific features in the training data.